

Modbus RTU (Astraada)

Sprawozdanie Jan Majerczyk, Wiktor Machelski wtorek 9.45

Cel ćwiczenia:

Celem ćwiczenia była realizacja protokołu Modbus RTU dla sterownika Astraada z użyciem programu Cscape.

Działanie:

Protokół Modbus opiera się na działaniu dwóch typów urządzeń master(maksymalnie jeden) kontroluje podległe urządzenia slave(maksymalnie 255) poprzez odpytywanie ich. Zaletami protokołu jest powszechnie stosowany przez to, że jest niedrogi oraz pozwala na wprowadzanie zmian do działającego już układu.

Przebieg ćwiczenia:

Ćwiczenie zaczęliśmy od wykonania początkowych konfiguracji oraz testów następnie wykonaliśmy ćwiczenie.

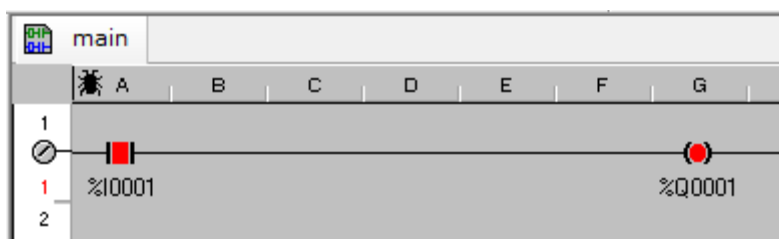
Pierwszym krokiem była konfiguracja programu Cscape. Czyli wprowadziliśmy adresy IP oraz wybraliśmy połączenie Ethernetem. W przypadku naszego ćwiczenia masterem jest PC ponieważ on odpytuje sterownik PLC (slave) o wciśnięty przycisk(wykrywane jest tylko zbocze nie liczy się jak długo jest wciśnięty przycisk). Sterownik jest podłączony do PC poprzez konwerter RS-485 na USB(PC ma wejścia USB ale komputery przemysłowe nie potrzebują konwertera(konwerter jest potrzebny przez różnice napięć)).

Do wykonania pierwszych testów musieliśmy napisać prosty program z użyciem języka LAD. Zaczeliśmy od dodania kontaktu oraz cewki(później będziemy sprawdzać jej stan w R2Sim).



Wszystkie bramki adresujemy(Modbus przyjmuje dwa typy danych pojedyncze bity oraz rejestry 16 bitowe(wejścia adresujemy jak IO(input) wyjścia jako Q(bo tak jest przyjęte) oraz miejsce w pamięci np. wartość logiczną cewki zapisujemy w M(memory))).

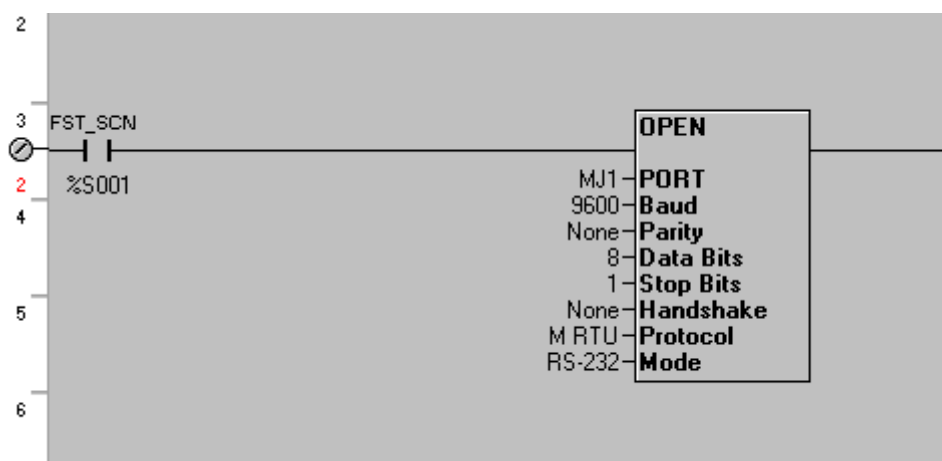
Następnie wgraliśmy program do sterownika. Gdy program został wgrany włączyliśmy opcje monitor przez co mogliśmy obserwować stan kontaktu oraz cewki.



Zauważyliśmy zmianę cewki przy zboczu narastającym z przycisku.

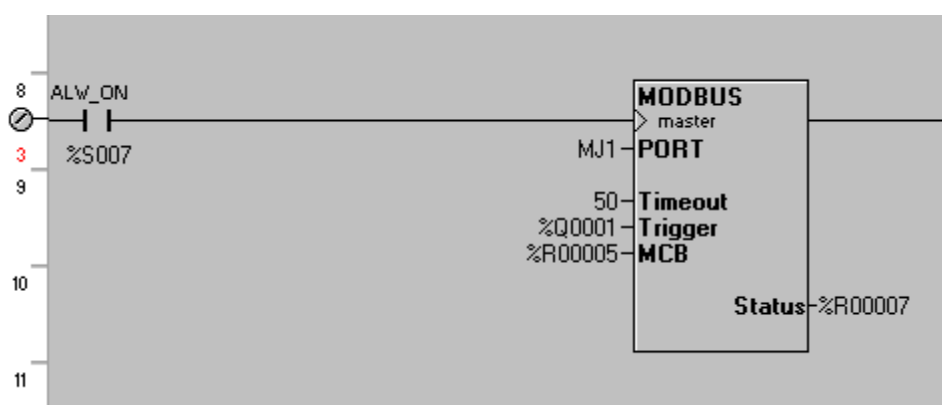
Kolejnym krokiem w testach była konfiguracja obsługi komunikacji modułu Modbus RTU, protokół Modbus korzysta z popularnej obecnie komunikacji szeregowej (popularniejsza od równoległej). Modbus w tym przypadku jest masterem.

Później dodaliśmy blok Open Comm Port, który tworzy kanał komunikacyjny do portu RS232 (regular standard (połączenie szeregowe)) w sterowniku.



Przypisaliśmy odpowiednie rejestry ustawiamy parametry i bity które odpytywaliśmy.

Później dodaliśmy blok Modbus Master.



Również ustawiliśmy odpowiednie rejestry.

Do wykonania testów potrzebne było nam 6 16-bitowych rejestrów (użyliśmy tagów czyli przypisaliśmy nazwy do nich).

%R00001	16-bit	MbusSlaveID
%R00002	16-bit	MbusCommand
%R00003	16-bit	MbusSlaveOffset
%R00004	16-bit	MbusDataLength
%R00005	16-bit	MbusReferenceType
%R00006	16-bit	MbusReferenceOffset
%R00007	16-bit	MbusStatus

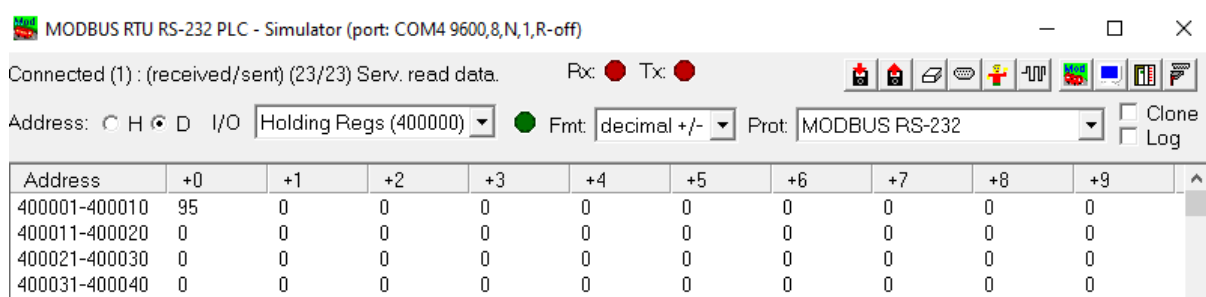
Ostatnimi krokami przed wykonaniem zadań były testy. Do tego wykorzystaliśmy program ModRsim2. Wykonaliśmy wszystkie kroki zgodnie z konspektem.

Test 1.

Test numer 1 polegał na odczytaniu stan pierwszej cewki slave, potrzebowaliśmy watch do rejestrów kontrolnych. Ustawiliśmy odpowiednie wartości w tabeli watch oraz Wybraliśmy odpowiednie parametry w Mod2Rsim. Zauważyliśmy zmiany. Niestety nie wykonałem screena testu nr 1.

Test 2.

Test numer 2 polegał na cyklicznym wysyłaniu do PC wartości logicznej licznika. Do tego potrzebowaliśmy boczka counter oraz dodaliśmy dodatkowy tag oraz zmieniliśmy parametry tagów. Na screenie poniżej można zobaczyć którąś z rzędu iteracje countera.



ZADANIA

Zadanie 1

Zadanie 1 polegało na zapytaniu slave o 16 cewek(od 17 do 32). Wykorzystaliśmy jedno polecenie Modbus. Do wykonania zadania zmieniliśmy wartości tagów w tabeli watch oraz dodaliśmy 16 tagów memory(każdy z nich ma w sobie wartość cewki).

Memory	Value	Type	Name
%R00002	1	INT	MbusCommand
%R00003	16	INT	MbusSlaveOffset
%R00004	16	INT	MbusDataLength
%R00005	76	INT	MbusReference...
%R00006	17	INT	MbusReference...
%R00007	2#0000000000000000	BIN_16	MbusStatus
%R00001	1	INT	MbusSlaveID
%M00001	ON	BOOL	
%M00002	ON	BOOL	
%M00003	ON	BOOL	
%M00004	ON	BOOL	
%M00005	ON	BOOL	
%M00006	OFF	BOOL	
%M00007	OFF	BOOL	
%M00008	OFF	BOOL	
%M00009	OFF	BOOL	
%M00010	OFF	BOOL	
%M00011	OFF	BOOL	
%M00012	OFF	BOOL	
%M00013	OFF	BOOL	
%M00014	ON	BOOL	
%M00015	ON	BOOL	
%M00016	ON	BOOL	

Address: ☐ H ☒ D I/O ☐ Coil Outputs (000000) ☐ Fmt: decimal +/- Prot: MODBUS RS-232																	
Address	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+10	+11	+12	+13	+14	+15	Total
000001-000016	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000
000017-000032	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	E01F
000033-000048	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000

Jak widać wartości modułu Modbus odpowiadają zadany wartości w watchu.

```

14:25:20.859 Read Output Coils from 16 for 16 bits.
14:25:20.859 TX:01 01 02 89 68 DF 82
14:25:22.280 RX:01 01 00 10 00 10
14:25:22.383 RX:3C 03
14:25:22.383 m_PDUSize is 256
14:25:22.383 Read Output Coils from 16 for 16 bits.
14:25:22.383 TX:01 01 02 89 68 DF 82

```

Pierwszy wiersz prezentuje ramkę zapytania RX czyli zapytanie z PLC do ModRsim2. Wartości określają kolejno adres(opisuje adres slave'a), funkcję(kod funkcji 1 ma odczytywać cewki), adres początkowy(dwa bajty jak widać pokazuje wartość 16 0010 w systemie szesnastkowym to 16), liczba cewek(dwa bajty również wartość tego wynosi 16).

Wiersz drugi prezentuje ramkę CRC

Zadanie 2

W zadaniu numer 2 ustawiliśmy pierwsze 5 cewek na wartość 1 z użyciem Modbus.

Polecenie numer 15 w Modbus służy do ustawienia wielu wartości cewek jednocześnie w urządzeniu slave (wykorzystuje: adres slave, kod funkcji, adres startowy, liczbę cewek, bajty cewek, ilość danych oraz sumę kontrolną). Tak jak w poprzednim ćwiczeniu odpowiednio zmodyfikowaliśmy tabele watch. Zmieniliśmy również adresy pamięci dla wartości cewek w celu uniknięcia błędów. Przebieg ćwiczenia widać w poniższych screenach.

Watch - JM_WM_Modbus.csp(253)

File

Memory	Value	Type	Name
%R00002	15	INT	MbusCommand
%R00003	0	INT	MbusSlaveOffset
%R00004	5	INT	MbusDataLength
%R00005	76	INT	MbusReference...
%R00006	17	INT	MbusReference...
%R00007	2#0000000000000000	BIN_16	MbusStatus
%R00001	1	INT	MbusSlaveID
%M00020	ON	BOOL	
%M00021	ON	BOOL	
%M00022	ON	BOOL	
%M00023	ON	BOOL	
%M00024	ON	BOOL	

Address	+0	+1	+2	+3	+4
000001-000016	1	1	1	1	1
000017-000032	0	0	0	0	0
000033-000048	0	0	0	0	0

```

14:28:40.511 RX:01 0F 00 00 00 05
14:28:40.616 RX:01 1F 2E 9E
14:28:40.616 m_PDUSize is 256
14:28:40.616 Write multiple outputs coils from 0 for 5 bits.
14:28:40.616 TX:01 0F 00 00 00 05 95 C8

```

TX01 oznacza że wiadomość została wysłana z mastera o adresie 01. 0F oznacza kod funkcji zapisującego wiele cewek, 0000 opisuje miejsce startu, 0005 zapisuje ilość cewek czyli 16. 95C8 oznacza potencjalny error.

Zadanie 3

Celem zadania numer 3 było przestanie wiadomości „AGH” w kodzie ASCII. Użyliśmy do tego funkcji Modbus numer 16, która służy do jednoczesnego zapisu wielu rejestrów 16 bitowych w slave. Format funkcji wygląda bardzo podobnie jak funkcji numer 15 ale

Watch - JM_WM_Modbus.csp(253)

File

Memory	Value	Type	Name
%R00002	16	INT	MbusCommand
%R00003	0	INT	MbusSlaveOffset
%R00004	5	INT	MbusDataLength
%R00005	8	INT	MbusReference...
%R00006	29	INT	MbusReference...
%R00007	2#0000000000000000	BIN_16	MbusStatus
%R00001	1	INT	MbusSlaveID
%M00001	OFF	BOOL	
%R00062	"I"	ASCII	
%R00060	"GA"	ASCII	
%R00061	"IH"	ASCII	

Address: ☐ H ☒ D I/O ☒ Fmt:

Address	+0	+1	+2	+3
400001-400010	A G	H .	..	x00 x00
400011-400020	x00 x00	x00 x00	x00 x00	x00 x00
400021-400030	x00 x00	x00 x00	x00 x00	x00 x00

```
14:32:43.996 RX:01 10 00 00 00 05
14:32:44.002 RX:0A 41 47 48 21 21
14:32:44.009 RX:21 00 00 00 00 CB
14:32:44.111 RX:38
14:32:44.111 m_PDUSize is 256
14:32:44.111 Write multiple Holding Registers from 0 for 5.
14:32:44.111 TX:01 10 00 00 00 05 00 0A
```

Zadanie 4

Celem zadania 4 było wywołanie błędu w module Modbus takich jak wystąpienie błędnego zakresu danych lub niepoprawnej funkcji Modbus.

Memory	Value	Type	Name
%R00002	2000	INT	MbusCommand
%R00003	0	INT	MbusSlaveOffset
%R00004	5	INT	MbusDataLength
%R00005	8	INT	MbusReference...
%R00006	29	INT	MbusReference...
%R00007	2#0000000000010010	BIN_16	MbusStatus
%R00001	1	INT	MbusSlaveID
%M00001	OFF	BOOL	
%R00062	"!"	ASCII	
%R00060	"GA"	ASCII	
%R00061	"IH"	ASCII	

Memory	Value	Type	Name
%R00002	1	INT	MbusCommand
%R00003	0	INT	MbusSlaveOffset
%R00004	265	INT	MbusDataLength
%R00005	8	INT	MbusReference...
%R00006	29	INT	MbusReference...
%R00007	2#0000000000100010	BIN_16	MbusStatus
%R00001	1	INT	MbusSlaveID
%M00001	OFF	BOOL	
%R00062	"!"	ASCII	
%R00060	"GA"	ASCII	
%R00061	"IH"	ASCII	